

EG2300 CPU Design Project

Project Specification

Version: February 8, 2026

Key Parameters

- **Datapath width:** 8-bit (register contents and ALU operations)
- **Instruction width:** 16-bit (each ROM word is a 16-bit instruction)
- **Instruction ROM:** 16 words $\times$ 16 bits (addresses 0–15)
- **Programme Counter (PC):** 4-bit, reset vector = 0
- **PC overflow:** sequential execution wraps modulo 16 (after 1111 goes to 0000), unless altered by control flow
- **Recipe parameters:** batch counts *A* and *B* (each 1–100) are defined by the ROM programme
- **Halt conditions:** STOP or FAULT or SYS_ERR; motor is forced off while halted
- **Execution modes:** Step and Run

1 Purpose

In this project you will design and implement a small CPU-based control system for a simple manufacturing station. The aim is to practise the complete digital design workflow: interpreting a specification, designing an architecture, implementing it in logic, verifying correct behaviour, and communicating design decisions.

You will build:

- A CPU that executes a programme stored in ROM.
- A small I/O subsystem (sensors/controls in, actuators/indicators out).
- A demonstrable control programme that operates a manufacturing station and handles faults safely.

2 Scenario

A legacy packaging line uses removable ROM cards to change the *packing recipe* between production runs. The mechanical hardware is fixed and intentionally simple for cost and reliability reasons. The controller is updated by swapping a ROM card rather than rewiring logic.

In this project, you will design the CPU and platform that run these ROM-based recipes, and demonstrate one working recipe on a simplified manufacturing station.

3 What you must deliver

3.1 Circuit implementation (Logisim)

You **MUST** submit a Logisim circuit containing:

- The CPU core (datapath + control).
- Instruction ROM (initialised with your programme).
- Input switches (representing sensors/operator controls).
- Output indicators (LEDs/bits representing actuators/indicators).
- Step/Run execution modes.

3.2 Team report and presentation/demo

You **MUST** submit a structured report and deliver a short demonstration of the CPU running your control programme, including fault handling.

Note

The assessment weightings and submission details are defined in the *Marking Criteria*. This document defines the *technical* requirements.

4 Fixed platform requirements (essential)

4.1 Data and instruction widths

- **Datapath width:** 8-bit (register contents and ALU operations are 8-bit).
- **Instruction width:** 16-bit (each ROM word is a 16-bit instruction).

4.2 Instruction ROM

- **ROM size:** 16 rows $\times$ 16-bit words
- **Addresses:** 0 to 15 (16 instructions total)

4.3 Programme counter (PC)

- **PC width:** 4-bit (addresses 0–15)
- **Reset behaviour:** on reset, **PC = 0** and outputs go to a safe state.
- **PC overflow behaviour:** sequential execution wraps modulo 16, unless altered by control flow.

4.4 Execution modes

You **MUST** implement both:

- **Step mode:** executes one instruction per button press.
- **Run mode:** executes continuously with a clock.

5 Factory function and station behaviour (batch diverter)

You will implement a **batch diverter packaging station**. Items arrive on a conveyor and must be routed to one of two packing lanes: **Gate A** or **Gate B**. The station runs a *recipe* stored in ROM:

- Send A items to Gate A,
- then send B items to Gate B,
- then repeat indefinitely, or *halt* depending on the software.

A line operator changes the packing recipe by swapping the ROM card. In this project, A and B must each be in the range **1–100**.

5.1 Required inputs (digital)

You **MUST** implement the following inputs:

- REBOOT: system reset. While REBOOT=1, the CPU is held in reset (PC = 0000, registers cleared, outputs safe).
- STOP: operator stop/hold. When STOP=1, the system is *halted/paused*.
- FAULT: non-emergency fault input (e.g., jam, guard open). When FAULT=1, the system is *halted/paused*.
- ITEM_PULSE: one conditioned pulse per item (one pulse corresponds to one item passing the counting point).

5.2 Required outputs (digital)

You **MUST** implement the following outputs:

- GATE_SEL: selects the active packing lane (0 = Gate A, 1 = Gate B).
- MOTOR_EN: conveyor motor enable.
- ALARM: general alarm output.

You **SHOULD** implement an additional status/debug output (e.g., STATUS[1:0] or STATUS[3:0]) to support demonstration and marking.

5.3 Platform halt, alarm, and safety rules (not negotiable)

To keep the design realistic and to fit within the 16-instruction ROM constraint, the platform enforces the following rules in hardware:

- The system is considered **halted** when STOP=1, FAULT=1, or an internal SYS_ERR=1.
- While halted, MOTOR_EN is forced to **0** (motor off), regardless of the programme.
- ALARM is asserted when FAULT=1 or SYS_ERR=1. (Normal STOP does not raise the alarm.)

- **SYS_ERR MUST** assert if an illegal/undefined instruction encoding is fetched. SYS_ERR is latched until REBOOT=1.

Note

Arithmetic overflow is *not* considered a system error in this project; 8-bit arithmetic wraps naturally unless your ISA appendix specifies otherwise.

5.4 Required behaviours (functional requirements)

Your ROM programme and platform **MUST** implement the following behaviours:

- **R1 — Safe reboot:** When REBOOT=1, the CPU state is cleared (PC = 0000, registers cleared) and outputs are safe: MOTOR_EN = 0, ALARM = 0, and a defined default for GATE_SEL (typically Gate A). When REBOOT returns to 0, execution begins from address 0000.
- **R2 — Normal running:** When not halted, the programme enables the conveyor (MOTOR_EN=1) and routes items according to the recipe.
- **R3 — Batch routing:** Each ITEM_PULSE causes the current batch counter to decrement by one. After exactly *A* pulses, GATE_SEL switches from Gate A to Gate B; after exactly *B* pulses, it switches back to Gate A; then the pattern repeats.
- **R4 — Stop/hold:** When STOP=1, the system halts and the motor is off. When STOP returns to 0 (and FAULT=0 and SYS_ERR=0), the system resumes from the point it paused.
- **R5 — Fault:** When FAULT=1, the system halts, the motor is off, and ALARM=1. When FAULT returns to 0 (and STOP=0 and SYS_ERR=0), the system resumes.
- **R6 — System error:** If SYS_ERR=1, the system halts with ALARM=1 and remains halted until REBOOT=1.
- **R7 — Observability:** Your design **MUST** provide clear observability for marking, including at least one debug/status indication (e.g., a status code for *running*, *stopped*, *fault*, *system error*).

6 CPU requirements

6.1 ISA compliance

Your CPU **MUST** implement the instruction set defined in the ISA specification (provided separately as an appendix). Your design **MUST** correctly fetch, decode, and execute those instructions.

6.2 Core components (minimum)

Your CPU **MUST** include:

- Register file (as specified by the ISA)
- ALU
- Instruction register and decoder/control logic
- Programme counter

- ROM interface (16-bit instruction word)
- A mechanism to read inputs and drive outputs (see Section 7)

7 I/O interface requirement (choose one)

To connect the CPU to the station signals, your team **MUST** select and implement **one** of the following I/O approaches:

- **Option A — Memory-mapped I/O:** Specific addresses read as inputs and/or write as outputs.
- **Option B — Dedicated I/O instruction(s):** A defined instruction reads inputs or writes outputs.
- **Option C — Register-mapped I/O:** Inputs appear in a dedicated register; outputs are driven from a dedicated output register.

Your report **MUST** justify your choice (see Section 8).

8 Required design decisions (AHEP4-aligned)

You **MUST** implement the base platform above, and in addition make **at least two** documented design decisions from the list below.

For each decision, your report **MUST** include:

- the requirement it addresses,
- at least two options considered,
- the chosen approach and trade-offs,
- evidence (tests and/or demo behaviour).

Choose **at least two**:

- **D1 — I/O interface strategy** (Section 7 Option A/B/C)
- **D2 — Halt and safety integration strategy** (how you implement STOP/FAULT/SYS_ERR halting, motor override, alarm behaviour, and resume behaviour)
- **D3 — Debug/observability strategy** (what you expose: status codes, state outputs, step/run behaviour, etc.)
- **D4 — Recipe and counting strategy** (how *A* and *B* are represented/loaded and how item pulses are counted within the 16-instruction limit)
- **D5 — Control implementation strategy** (hardwired control vs microcoded control, if you choose to implement it)

9 Optional extensions

Extensions are optional and should only be attempted after meeting all base requirements. Examples:

- **Parameter inputs for A and B:** allow the operator to choose counts via switches (loaded on reboot or when leaving STOP).
- **Richer outputs:** replace GATE_SEL with separate GATE_A/GATE_B actuation pulses, or add a multi-bit STATUS code.
- **Pulse conditioning:** implement edge detection or pulse stretching for ITEM_PULSE and justify the trade-offs.
- **Additional non-emergency faults:** e.g., multiple fault sources combined into FAULT with a visible fault code.
- **Simple low-power feature:** reduce switching activity when halted (e.g., via a clock-enable strategy).

10 Verification and definition of done

Definition of Done

Your submission is considered complete when you can demonstrate the acceptance tests in Section 10.1 and provide evidence in your report.

10.1 Acceptance tests (minimum)

You **MUST** demonstrate (in Step and/or Run mode):

1. **Reboot test:** with REBOOT=1, PC is 0000, registers are cleared, MOTOR_EN=0, ALARM=0, and GATE_SEL is in a defined default state. Releasing REBOOT starts execution from address 0000.
2. **Batch routing test:** demonstrate the recipe A/B by showing GATE_SEL switches after exactly A item pulses and then after exactly B item pulses, repeating.
3. **Stop/hold test:** assert STOP=1 during operation. Show that the system halts and MOTOR_EN is forced to 0. Release STOP and show the system resumes.
4. **Fault test:** assert FAULT=1 during operation. Show that the system halts, MOTOR_EN=0, and ALARM=1. Clear FAULT and show the system resumes with ALARM=0.
5. **System error test:** demonstrate that an illegal/undefined instruction causes SYS_ERR (or equivalent indication) and halts the system with ALARM=1, and that only REBOOT clears this condition.
6. **Observability test:** explain and demonstrate your status/debug output(s) (e.g., running/stopped/-fault/system error).

10.2 Evidence format

Your report **SHOULD** include:

- A short test table (test name → input sequence → expected output → observed output).
- Screenshots or timing-style traces where helpful.

- A brief explanation of any limitations caused by the 16-instruction ROM constraint.

11 Report structure (required headings)

Use the following headings to keep reports uniform:

1. System overview (what you built, block diagram)
2. CPU architecture (datapath/control, key components)
3. I/O mapping and station interface (signals and how CPU accesses them)
4. ROM programme overview (what each instruction region/state does)
5. Design decisions (at least two, using the required format)
6. Verification (acceptance tests and results)
7. Limitations and improvements (what you would do with more ROM/time)

12 Presentation/demo expectations

Your demo **MUST** show:

- Step mode execution for a short segment (so behaviour is understandable).
- Run mode continuous operation.
- The batch recipe switching (*A* items to Gate A, then *B* items to Gate B, repeating).
- STOP pause/resume behaviour (motor forced off while stopped).
- FAULT behaviour (alarm asserted, motor off) and recovery when the fault clears.
- A clear explanation of your observability/debug outputs (and SYS_ERR behaviour if demonstrated).

13 Implementation notes

- **ROM file format:** your ROM is 16 lines; line *n* is address *n*; each line is a 16-bit binary word.
- **ITEM_PULSE:** assume this is already conditioned (one clean pulse per item). Do not spend ROM space on debouncing unless you choose it as an extension.
- **Halt/safety logic:** implement `HALT = STOP OR FAULT OR SYS_ERR`. While halted, force `MOTOR_EN=0` and assert `ALARM = FAULT OR SYS_ERR`.
- **Tiny ROM constraint:** 16 instructions is intentionally restrictive. Write a compact loop and reuse branches where possible.
- **Uniform signal naming:** keep circuit labels consistent with this document.

A ISA Specification (to be attached)

Attach your ISA specification here.

Note

Option 1: paste the ISA text into this appendix.

Option 2: include a separate PDF:

- Upload the ISA PDF to Overleaf (e.g., isa.pdf)
- Uncomment the lines below and compile.